National University of Computer and Emerging Sciences
(FAST-NUCES)
PROJECT PROPOSAL

OPERATING SYSTEM

Bank and ATM System

Instructor: Sir Ubaidullah

Group Members:

- Ghulam Mujtaba Qureshi – 24K-0535
- Ali Rooman – 24K-0792
- Hanzala Ramzan – 23K-2027

Introduction

In modern banking systems, many users perform transactions at the same time. If multiple operations access the same account simultaneously, it can cause problems such as incorrect balances or race conditions. To avoid these issues, synchronization techniques are required.

This project implements a **Concurrent Banking Transaction System using multithreading in C**. Multiple threads simulate clients performing banking operations such as deposits, withdrawals, and transfers. Synchronization mechanisms like **mutex locks and semaphores** are used to ensure that shared account data is accessed safely.

Objectives

The main objectives of this project are:

- To simulate banking transactions using multiple threads
- To ensure safe access to shared accounts using mutex
- To control concurrent transactions using semaphores
- To maintain transaction logs
- To verify that account balances remain consistent

System Overview

The system creates several bank accounts and multiple threads that perform transactions simultaneously. Each thread represents a client performing operations like deposit, withdrawal, or transfer.

Since multiple threads may access the same account, **mutex locks are used to protect critical sections**, ensuring that only one thread updates an account balance at a time. Transaction details are also logged so that system activity can be monitored.

The system runs as a **console-based application**, where transaction activity and account updates are displayed in the terminal.

Tools and Technologies

- **Programming Language:** C
- **Library:** POSIX pthread
- **Synchronization:** Mutex and Semaphore
- **Platform:** Linux / Ubuntu
- **Compiler:** GCC

Expected Output

The system will simulate multiple banking transactions running concurrently. The program will display transaction execution, account balance updates, and transaction logs in the console while multiple threads perform operations simultaneously. This helps observe synchronization and balance consistency.

Future Enhancement

In future versions, a **GUI can be added** to visually display accounts, transactions, and logs. This would make the system easier to monitor and interact with.

Conclusion

This project demonstrates important operating system concepts such as **multithreading, synchronization, and shared resource management**. By using mutexes and semaphores, the system ensures safe and consistent execution of concurrent banking transactions.